RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



RAJALAKSHMI
ENGINEERING COLLEGE
An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

CS23331 - DESIGN AND ANALYSIS OF ALGORITHMS

Laboratory Record Note Book

Name	:	Janapriya D
Year / Branch / Section	:	2nd / CSE / M
University Register No.	:	2116250701260
College Roll No.	:	250701260
Semester	:	3rd
Academic Year	:	2026-2027

INDEX PAGE

Exp.No	Name of the Experiment	Date of Completion
1	Counter Method: Loop with s += i	
2	Counter Method: Loop with break	
3	Counter Method: Factors of a number	
4	Counter Method: Nested log loops	
5	Counter Method: Reverse a number	
6	D&C: Count zeroes in array	
7	D&C: Majority element	
8	D&C: Floor of x in sorted array	
9	D&C: Pair with sum x	
10	D&C: Quick Sort	
11	DP: Playing with Numbers	
12	DP: Playing with Chessboard	
13	DP: Longest Common Subsequence	
14	DP: Longest Non-decreasing Subsequence	
15	CP: Find Duplicate in Array	
16	CP: Intersection of Sorted Arrays	
17	CP: Pair with Difference k	
18	Greedy: Minimum Coin Change	
19	Greedy: Cookies and Children	
20	Greedy: Burgers and Calories	
21	Greedy: Maximize sum of arr[i]*i	
22	Greedy: Minimum Sum of Products	

CS23331-Design and Analysis of Algorithms

Problem 1: Finding Complexity using Counter Method

Completion requirements

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function (int n)
{
    int i= 1;
    int s =1;
    while(s <= n)
    {
         i++;
         s += i;
     }
}
```

Note: No need of counter increment for declarations and scanf() and printf() statements. count variable

Input:
A positive Integer n

Output:
Print the value of the counter variable

For example:

Input	Result
9	12

ANSWER:

```
#include<stdio.h>

int main(){

   int n;

   scanf("%d",&n);

   int i=1;

   int s=1;

   int count=2;

   while(s<=n){
```

```
    count++;
    i++;
    count++;
    s+=i;
    count++;
  }
  int answer=count+1;
  printf("%d",answer);
  return 0;
}
```

Input	Expected	Got	
	9	12	12
	4	9	9

Problem 2: Finding Complexity using Counter method

```
Convert the following algorithm into a program and find its time complexity using the
counter method.
void func(int n)
{
    if(n==1)
    {
      printf("*");
    }
    else
    {
     for(int i=1; i<=n; i++)
     {
       for(int j=1; j<=n; j++)
       {
          printf("*");
          printf("*");
          break;
       }
```

```
    }
  }
}
```

Note: No need of counter increment for declarations and scanf() and printf() statements. count variable

Input:

A positive Integer n

Output:

Print the value of the counter variable

ANSWER:

```
#include<stdio.h>

int main(){

   int n;

   scanf("%d",&n);

   int count=0;

   count++;

   if(n==1)

   {

    printf("*");

   }

   else

   {

    for(int i=1; i<=n; i++)

    {

     count++;

     for(int j=1; j<=n; j++)

     {

       count++;

       count++;

       count++;

       break;

     }

     count++;;

    }

    count++;

  }
```

```
  printf("%d",count);
  return 0;
}
```

Input	Expected	Got	
	2	12	12
	1000	5002	5002
	143	717	717

Problem 3: Finding Complexity using Counter Method

Convert the following algorithm into a program and find its time complexity using counter method.

```
 Factor(num) {
 {
    for (i = 1; i <= num;++i)
    {
     if (num % i== 0)
        {
          printf("%d ", i);
        }
     }
  }
```

Note: No need of counter increment for declarations and scanf() and counter variable printf() statement.

Input:
A positive Integer n
Output:
Print the value of the counter variable

ANSWER:

```
#include <stdio.h>
int main()
{
   int num;
   scanf("%d",&num);
   int count = 0;
   int i;
   for(i = 1; i <= num; i++)
```

```
  {
    count++;
    if(num % i == 0)
    {
      count++;
    }
    count++;
  }
  count++;
  printf("%d",count);
  return 0;
}
```

Input	Expected	Got	
	12	31	31
	25	54	54
	4	12	12

Problem 4: Finding Complexity using Counter Method

Convert the following algorithm into a program and find its time complexity using counter method.

```
void function(int n)
{
    int c= 0;
    for(int i=n/2; i<n; i++)
        for(int j=1; j<n; j = 2 * j)
            for(int k=1; k<n; k = k * 2)
                c++;
}
```

Note: No need of counter increment for declarations and scanf() and printf() statements. count variable

Input:
A positive Integer n
Output:
Print the value of the counter variable

ANSWER:

```
#include<stdio.h>
int main(){
   int n;
   scanf("%d",&n);
   int c=0;
   int count=2;
   for(int i=n/2;i<n;i++){
      count++;
      for(int j=1;j<n;j=2*j){
         count++;
         for(int k=1;k<n;k=k*2){
            count++;
            c++;
            count++;
         }
         count++;
      }
      count++;
   }
   printf("%d",count);
   return 0;
}
```

Input	Expected	Got	
	4	30	30
	10	212	212

Problem 5: Finding Complexity using counter method

Convert the following algorithm into a program and find its time complexity using counter method.

```
void reverse(int n)
{
   int rev = 0, remainder;
   while (n != 0)
    {
        remainder = n % 10;
        rev = rev * 10 + remainder;
        n/= 10;

    }
print(rev);
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

ANSWER:

```
#include<stdio.h>

int main(){
    int n;
    scanf("%d",&n);
    int rev=0,rem;
    int count=0;
    while(n!=0){
        count++;
        rem=n%10;
        count++;
        rev=rev*rem;
        count++;
        n=n/10;
        count++;
    }
    count+=3;
    printf("%d",count);
    return 0;
}
```

Input	Expected	Got	
	12	11	11
	1234	19	19

1-G-Coin Problem

Write a program to take value V and we want to make change for V Rs, and we have infinite supply of each of the denominations in Indian currency, i.e., we have infinite supply of { 1, 2, 5, 10, 20, 50, 100, 500, 1000} valued coins/notes, what is the minimum number of coins and/or notes needed to make the change.

Input Format:

Take an integer from stdin.

Output Format:

print the integer which is change of the number.

Input :

64

Output:

4

Explanaton:

We need a 50 Rs note and a 10 Rs note and two 2 rupee coins.

ANSWER:

```
#include <stdio.h>

int main() {

    int V, count = 0;

    int coins[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};

    int n = 9;

    scanf("%d", &V);

    for (int i = 0; i < n; i++) {

        count += V / coins[i];

        V = V % coins[i];
```

```
    }

    printf("%d", count);

    return 0;

}
```

	Input	Expected	Got	
	49	5	5	

Passed all tests!

2-G-Cookies Problem

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

Input:

3

1 2 3

2

1 1

Output:

1

Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.

And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.

You need to output 1.

Constraints:

1 <= g.length <= 3 * 10^4

0 <= s.length <= 3 * 10^4

1 <= g[i], s[j] <= 2^31 – 1

ANSWER:

```
#include <stdio.h>
int main() {
    int n, m, temp;
    scanf("%d", &n);
    int g[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &g[i]);
    }
    scanf("%d", &m);
    int s[m];
    for (int i = 0; i < m; i++) {
        scanf("%d", &s[i]);
    }
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (g[i] > g[j]) {
                temp = g[i];
                g[i] = g[j];
                g[j] = temp;
            }
        }
    }
    for (int i = 0; i < m - 1; i++) {
        for (int j = i + 1; j < m; j++) {
            if (s[i] > s[j]) {
```

```
                temp = s[i];
                s[i] = s[j];
                s[j] = temp;
            }
        }
    }
    int i = 0, j = 0;
    int count = 0;
    while (i < n && j < m) {
        if (s[j] >= g[i]) {
            count++;
            i++;
            j++;
        } else {
            j++;
        }
    }
    printf("%d", count);
    return 0;
}
```

	Input	Expected	Got	
	2 1 2 3 1 2 3	2	2	

Passed all tests!

3-G-Burger Problem

A person needs to eat burgers. Each burger contains a count of calorie. After eating the burger, the person needs to run a distance to burn out his calories.
If he has eaten *i* burgers with c calories each, then he has to run at least $3^i * c$ kilometers to burn out the calories. For example, if he ate 3
burgers with the count of calorie in the order: [1, 3, 2], the kilometers he needs to run are $(3^0 * 1) + (3^1 * 3) + (3^2 * 2) = 1 + 9 + 18 = 28$.
But this is not the minimum, so need to try out other orders of consumption and choose the minimum value. Determine the minimum distance
he needs to run. Note: He can eat burger in any order and use an efficient sorting algorithm.Apply greedy approach to solve the problem.

Input Format

First Line contains the number of burgers
Second line contains calories of each burger which is n space-separate integers

Output Format

Print: Minimum number of kilometers needed to run to burn out the calories

Sample Input

3
5 10 7

Sample Output
76

For example:

Test	Input	Result
Test Case 1	3 1 3 2	18

ANSWER:

```
#include <stdio.h>
#include <math.h>
int main()
{
   int n, i, j, temp;
   int sum=0;
   scanf("%d", &n);
   int calories[n];
   for (i = 0; i < n; i++)
   {
      scanf("%d", &calories[i]);
   }
   for (i = 0; i < n ; i++)
   {
      for (j = i+1; j < n; j++)
      {
         if (calories[i] < calories[j])
         {
            temp = calories[i];
            calories[i] = calories[j];
            calories[j] = temp;
         }
      }
```

```
  }
   for (i = 0; i < n; i++)
   {
   sum=sum+pow(n,i)*calories[i];
   }
   printf("%d", sum);
   return 0;
}
```

	Test	Input	Expected	Got	
	Test Case 1	3 1 3 2	18	18	
	Test Case 2	4 7 4 9 6	389	389	
	Test Case 3	3 5 10 7	76	76	

Passed all tests!

4-G-Array Sum max problem

Given an array of N integer, we have to maximize the sum of arr[i] * i, where i is the index of the element (i = 0, 1, 2, ..., N).Write an algorithm based on Greedy technique with a Complexity O(nlogn).

Input Format:

First line specifies the number of elements-n

The next n lines contain the array elements.

Output Format:

Maximum Array Sum to be printed.

Sample Input:

5
2 5 3 4 0

Sample output:
40

ANSWER:

```
#include <stdio.h>

int main() {
    int n, temp;
    long long sum = 0;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (arr[i] > arr[j]) {
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
    for (int i = 0; i < n; i++) {
        sum += arr[i] * i;
    }
```

```
    printf("%lld", sum);

    return 0;

}
```

	Input	Expected	Got	
	5 2 5 3 4 0	40	40	
	10 2 2 2 4 4 3 3 5 5 5	191	191	
	2 45 3	45	45	

Passed all tests!

5-G-Product of Array Elements-Minimum

Given two arrays array_One[] and array_Two[] of same size N. We need to first rearrange the arrays such that the sum of the product of pairs(1 element from each) is minimum. That is SUM (A[i] * B[i]) for all i is minimum.

For example:

Input	Result
3 1 2 3 4 5 6	28

ANSWER:

```
#include <stdio.h>
int main() {
    int n, temp;
    int sum = 0;
    scanf("%d", &n);
    int A[n], B[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &A[i]);
    }
    for (int i = 0; i < n; i++) {
        scanf("%d", &B[i]);
    }
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (A[i] > A[j]) {
                temp = A[i];
                A[i] = A[j];
                A[j] = temp;
            }
        }
    }
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (B[i] < B[j]) {
                temp = B[i];
```

```
            B[i] = B[j];
            B[j] = temp;
        }
      }
    }
    for (int i = 0; i < n; i++) {
        sum += A[i] * B[i];
    }
    printf("%d", sum);
    return 0;
}
```

	Input	Expected	Got	
	3 1 2 3 4 5 6	28	28	
	4 7 5 1 2 1 3 4 1	22	22	
	5 20 10 30 10 40 8 9 4 3 10	590	590	

Passed all tests!

1-Number of Zeros in a Given Array

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format
First Line Contains Integer m – Size of array
Next m lines Contains m numbers – Elements of an array

Output Format
First Line Contains Integer – Number of zeroes present in the given array.

ANSWER:

```
#include <stdio.h>

int main()

{

    int m, i, low, high, mid, count;

    scanf("%d", &m);

    int a[m];

    for(i = 0; i < m; i++)

        scanf("%d", &a[i]);

    low = 0;

    high = m - 1;

    while(low <= high)

    {

        mid = (low + high) / 2;

        if(a[mid] == 1)

            low = mid + 1;

        else
```

```
            high = mid - 1;
        }
        count = m - low;
        printf("%d", count);
        return 0;
}
```

	Input	Expected	Got	
	5 1 1 1 0 0	2	2	
	10 1 1 1 1 1 1 1 1 1 1	0	0	
	8 0 0 0 0 0 0 0 0	8	8	
	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	

2-Majority Element

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

ANSWER:

```
#include <stdio.h>

int main()

{

   int n, i, j, count, maxCount = 0, majority;

   scanf("%d", &n);

   int nums[n];

   for(i = 0; i < n; i++)

      scanf("%d", &nums[i]);
```

```
    for(i = 0; i < n; i++)
    {
        count = 0;
        for(j = 0; j < n; j++)
        {
            if(nums[i] == nums[j])
                count++;
        }
        if(count > maxCount)
        {
            maxCount = count;
            majority = nums[i];
        }
    }
    printf("%d", majority);
    return 0;
}
```

	Input	Expected	Got	
	3 3 2 3	3	3	

Passed all tests!

3-Finding Floor Value

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array
Next n lines Contains n numbers – Elements of an array
Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

ANSWER:

```
#include <stdio.h>

int main()

{

   int n, i, x;

   int low = 0, high, mid;

   int floor = -1;

   scanf("%d", &n);

   int a[n];

   for(i = 0; i < n; i++)

      scanf("%d", &a[i]);

   scanf("%d", &x);

   high = n - 1;

   while(low <= high)

   {

      mid = (low + high) / 2;

      if(a[mid] == x)

      {

         floor = a[mid];

         break;

      }

      else if(a[mid] < x)

      {

         floor = a[mid];
```

```
            low = mid + 1;
        }
        else
        {
            high = mid - 1;
        }
    }
    printf("%d", floor);
    return 0;
}
```

	Input	Expected	Got	
	6 1 2 8 10 12 19 5	2	2	
	5 10 22 85 108 129 100	85	85	
	7 3 5 7 9 11 13 15 10	9	9	

Passed all tests!

4-Two Elements sum to x

Problem Statement:
Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as “No”.
Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array
Next n lines Contains n numbers – Elements of an array
Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1
Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value “x”)

ANSWER:

```
#include <stdio.h>

int search(int a[], int left, int right, int x)

{

   if (left >= right)

      return 0;

   int sum = a[left] + a[right];

   if (sum == x)

   {

      printf("%d\n", a[left]);

      printf("%d\n", a[right]);

      return 1;

   }

   else if (sum < x)

   {
```

```
        return search(a, left + 1, right, x);
    }
    else
    {
        return search(a, left, right - 1, x);
    }
}
int main()
{
    int n, i, x, result;
    scanf("%d", &n);
    int a[n];
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    scanf("%d", &x);
    result = search(a, 0, n - 1, x);
    if (result == 0)
        printf("No");
    return 0;
}
```

	Input	Expected	Got	
	4 2 4 8 10 14	4 10	4 10	
	5 2 4 6 8 10 100	No	No	

Passed all tests!

5-Implementation of Quick Sort

Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:

Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

ANSWER:

```
#include <stdio.h>

void quicksort(int a[], int low, int high)

{

   int i, j, pivot, temp;

   if(low < high)

   {

      pivot = a[low];

      i = low;

      j = high;


      while(i < j)

      {

         while(a[i] <= pivot && i < high)
```

```
            i++;
        while(a[j] > pivot)
            j--;
        if(i < j)
        {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
      }
      temp = a[low];
      a[low] = a[j];
      a[j] = temp;
      quicksort(a, low, j - 1);
      quicksort(a, j + 1, high);
   }
}
int main()
{
   int n, i;
   int a[100];
   scanf("%d", &n);
   for(i = 0; i < n; i++)
      scanf("%d", &a[i]);
   quicksort(a, 0, n - 1);
   for(i = 0; i < n; i++)
      printf("%d ", a[i]);
   return 0;
}
```

	Input	Expected	Got	
	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	
	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	
	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	

Passed all tests!

Dynamic Programming

1-DP-Playing with Numbers

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram term, so he gave Sita a positive integer ‘n’ and two numbers 1 and 3. He asked her to find the possible ways by which the number n can be represented using 1 and 3.Write any efficient algorithm to find the possible ways.

Example 1:

Input: 6
Output:6
Explanation: There are 6 ways to 6 represent number with 1 and 3

1+1+1+1+1+1
3+3
1+1+1+3
1+1+3+1
1+3+1+1
3+1+1+1

Input Format
First Line contains the number n

Output Format

Print: The number of possible ways ‘n’ can be represented using 1 and 3

Sample Input

6

Sample Output

6

ANSWER:

```
#include <stdio.h>

int main() {

   int n;
```

```
    scanf("%d", &n);
    long long dp[1001];
    dp[0] = 1;
    dp[1] = 1;
    dp[2] = 1;
    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 3];
    }
    printf("%lld", dp[n]);
    return 0;
}
```

	Input	Expected	Got	
✔	6	6	6	✔
✔	25	8641	8641	✔
✔	100	24382819596721629	24382819596721629	✔

Passed all tests! ✔

2-DP-Playing with chessboard

Ram is given with an n*n chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

Example:
Input
3
1 2 4
2 3 4
8 7 1
Output:

19

Explanation:
Totally there will be 6 paths among that the optimal is
Optimal path value:1+2+8+7+1=19

Input Format
First Line contains the integer n
The next n lines contain the n*n chessboard values

Output Format

Print Maximum monetary value of the path

ANSWER:

```
#include <stdio.h>
int main() {
    int n;
    scanf("%d", &n);
    int a[100][100];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
        }
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == 0 && j == 0) {
                continue;
            }
            if (i == 0) {
                a[i][j] = a[i][j] + a[i][j - 1];
            }
```

```
            else if (j == 0) {
                a[i][j] = a[i][j] + a[i - 1][j];
            }
            else {
                if (a[i - 1][j] > a[i][j - 1])
                    a[i][j] = a[i][j] + a[i - 1][j];
                else
                    a[i][j] = a[i][j] + a[i][j - 1];
            }
        }
    }
    printf("%d", a[n - 1][n - 1]);

    return 0;
}
```

	Input	Expected	Got	
✔	3 1 2 4 2 3 4 8 7 1	19	19	✔
✔	3 1 3 1 1 5 1 4 2 1	12	12	✔
✔	4 1 1 3 4 1 5 7 8 2 3 4 6 1 6 9 0	28	28	✔

Passed all tests! ✔

3-DP-Longest Common Subsequence

Given two strings find the length of the common longest subsequence(need not be contiguous) between the two.

Example:

s1: ggtabe

s2: tgatasb

s1	a	g	**g**	**t**	**a**	**b**	
s2	**g**	x	**t**	**x**	**a**	**y**	**b**

The length is 4

Solveing it using Dynamic Programming

For example:

Input	Result
aab azb	2

ANSWER:

```
#include <stdio.h>
#include <string.h>
int main() {
    char s1[100], s2[100];
    int dp[100][100];
    scanf("%s", s1);
    scanf("%s", s2);
    int n = strlen(s1);
    int m = strlen(s2);
```

```
    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= m; j++) {
            if (i == 0 || j == 0) {
                dp[i][j] = 0;
            }
            else if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            }
            else {
                if (dp[i - 1][j] > dp[i][j - 1])
                    dp[i][j] = dp[i - 1][j];
                else
                    dp[i][j] = dp[i][j - 1];
            }
        }
    }
    printf("%d", dp[n][m]);
    return 0;
}
```

	Input	Expected	Got	
✔	aab azb	2	2	✔
✔	ABCD ABCD	4	4	✔

Passed all tests! ✔

4-DP-Longest non-decreasing Subsequence

Problem statement:

Find the length of the Longest Non-decreasing Subsequence in a given Sequence.

Eg:

Input:9

Sequence:[-1,3,4,5,2,2,2,2,3]

the subsequence is [-1,2,2,2,2,3]

Output:6

ANSWER:

```
#include <stdio.h>
int main()
{
   int n;
   int a[100];
   int dp[100];
   int i, j;
   int max = 1;
   scanf("%d", &n);
   for (i = 0; i < n; i++)
   {
      scanf("%d", &a[i]);
      dp[i] = 1;
   }
   for (i = 1; i < n; i++)
   {
```

```
        for (j = 0; j < i; j++)
        {
            if (a[i] >= a[j])
            {
                if (dp[i] < dp[j] + 1)
                {
                    dp[i] = dp[j] + 1;
                }
            }
        }
        if (dp[i] > max)
        {
            max = dp[i];
        }
    }
    printf("%d", max);
    return 0;
}
```

	Input	Expected	Got	
✔	9 -1 3 4 5 2 2 2 2 3	6	6	✔
✔	7 1 2 2 4 5 7 6	6	6	✔

Passed all tests! ✔

1-Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

ANSWER:

```
#include <stdio.h>
int main()
{
   int n, i, j;
   scanf("%d", &n);
   int a[n];
   for (i = 0; i < n; i++)
   {
      scanf("%d", &a[i]);
   }
```

```
    for (i = 0; i < n; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (a[i] == a[j])
            {
                printf("%d", a[i]);
                return 0;
            }
        }
    }
    return 0;
}
```



	Input	Expected	Got	
✔	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✔
✔	5 1 2 3 4 4	4	4	✔
✔	5 1 1 2 3 4	1	1	✔

Passed all tests! ✔

2-Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

ANSWER:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
   int n, i, x;
   scanf("%d", &n);
   int a[n];
   for (i = 0; i < n; i++)
   {
      scanf("%d", &a[i]);
   }
   for (i = 0; i < n; i++)
   {
      x = abs(a[i]);
      if (a[x - 1] < 0)
      {
         printf("%d", x);
         return 0;
      }
```

```
        a[x - 1] = -a[x - 1];
    }
    return 0;
}
```



	Input	Expected	Got	
✔	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✔
✔	5 1 2 3 4 4	4	4	✔
✔	5 1 1 2 3 4	1	1	✔

Passed all tests! ✔

3-Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array
2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

ANSWER:

```
int main()
{
    int T;
    scanf("%d", &T);
    while (T--)
    {
        int n1, n2;
        scanf("%d", &n1);
        int a[n1];
```

```
for (int i = 0; i < n1; i++)
{
    scanf("%d", &a[i]);
}
scanf("%d", &n2);
int b[n2];
for (int i = 0; i < n2; i++)
{
    scanf("%d", &b[i]);
}
int i = 0, j = 0;
while (i < n1 && j < n2)
{
    if (a[i] == b[j])
    {
        printf("%d ", a[i]);
        i++;
        j++;
    }
    else if (a[i] < b[j])
    {
        i++;
    }
    else
    {
        j++;
    }
}
printf("\n");
```

```
    }
    return 0;
}
```

	Input	Expected	Got	
✔	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✔
✔	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✔

Passed all tests! ✔

4-Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array
2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

ANSWER:

```
#include <stdio.h>
int main()
{
   int T;
   scanf("%d", &T);
   while (T--)
   {
      int n1, n2;
      scanf("%d", &n1);
      int a[n1];
      for (int i = 0; i < n1; i++)
      {
         scanf("%d", &a[i]);
      }
      scanf("%d", &n2);
      int b[n2];
      for (int i = 0; i < n2; i++)
```

```
        {
            scanf("%d", &b[i]);
        }
        int i = 0;
        int j = 0;

        while (i < n1 && j < n2)
        {
            if (a[i] == b[j])
            {
                printf("%d ", a[i]);
                i++;
                j++;
            }
            else if (a[i] < b[j])
            {
                i++;
            }
            else
            {
                j++;
            }
        }
        printf("\n");
    }
    return 0;
}
```



	Input	Expected	Got	
✔	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✔
✔	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✔

Passed all tests! ✔

5-Pair with Difference-O(n^2)Time Complexity,O(1) Space Complexity

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3	1

Input	Result
1 3 5 4	

ANSWER:

```
#include <stdio.h>
int main()
{
    int n, k;
    scanf("%d", &n);
    int a[n];
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }
    scanf("%d", &k);
    int i = 0;
    int j = 1;
    while (j < n)
    {
        int difference = a[j] - a[i];
        if (difference == k && i != j)
        {
            printf("1");
            return 0;
        }
        else if (difference < k)
        {
```

```
            j++;
        }
        else
        {
            i++;
        }
        if (i == j)
        {
            j++;
        }
    }
    printf("0");
    return 0;
}
```

	Input	Expected	Got	
✔	3 1 3 5 4	1	1	✔
✔	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✔
✔	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✔
✔	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✔

Passed all tests! ✔

6-Pair with Difference -O(n) Time Complexity,O(1) Space Complexity

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

ANSWER:

```
#include <stdio.h>
int main()
{
   int n, k;
   scanf("%d", &n);
```

```
int a[n];
for (int i = 0; i < n; i++)
{
   scanf("%d", &a[i]);
}
scanf("%d", &k);
int i = 0;
int j = 1;
while (j < n)
{
   if (a[j] - a[i] == k && i != j)
   {
      printf("1");
      return 0;
   }
   else if (a[j] - a[i] < k)
   {
      j++;
   }
   else
   {
      i++;
      if (i == j)
      {
         j++;
      }
   }
}
printf("0");
```

```
    return 0;
}
```

	Input	Expected	Got	
✔	3 1 3 5 4	1	1	✔
✔	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✔
✔	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✔
✔	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✔

Passed all tests! ✔